

DSEM-NIDS: Enhanced Network Intrusion Detection System Using Deep Stacking Ensemble Model

LOREEN MAHMOUD ¹, MADHUSANKA LIYANAGE ² (Senior Member, IEEE), JITIN SINGLA ^{1,3},
AND SUGATA GANGOPADHYAY ¹

¹Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee 247667, India

²Network Softwarization and Security Labs (NetsLab), School of Computer Science, University College Dublin, D04 V1W8 Dublin, Ireland

³Department of Biosciences and Bioengineering, Indian Institute of Technology Roorkee, Roorkee 247667, India

CORRESPONDING AUTHOR: MADHUSANKA LIYANAGE (email:madhusanka@ucd.ie).

This work was supported in part by European Union through the ENSURE-6G Project under Grant 101182933, in part by Science Foundation Ireland through CONNECT phase 2 under Grant 13/RC/2077_P2, and in part by Information Security Education and Awareness (ISEA) Phase-3 Project funded by MeitY, Govt. of India.

ABSTRACT The need to deploy a network intrusion detection system (NIDS) is essential and has become increasingly necessary for every network, regardless whether it is wired, wireless, or hybrid, and its purpose is commercial, medical, defense, or social. Since the amount of data transfer over the Internet increases every year, using a single model as an IDS to secure the network cannot be considered enough as it may have many problems like high bias or high variance, which lead to high rates of false negatives and false positives. In this article, we propose an ensemble learning-based NIDS (DSEM-NIDS); this system is a deep-stacking model with a nested structure that has the ability to score a high performance with low false positive and low false negative rates. Four datasets are used as a benchmark to evaluate the proposed model: The 5G-NIDD, UNR-IDD, N-BaIoT, and NSL-KDD datasets. The results show that the proposed deep stacking model is robust, has good scalability, has the ability to distinguish between classes, and has the flexibility to adapt to different input data. It also performs better than other used models.

INDEX TERMS Deep stacking, ensemble learning, machine learning, network intrusion detection system.

I. INTRODUCTION

With the significantly growing network threats, there is always a huge need for methods to protect assets and prevent dangerous events. One of the most effective methods used in this domain is the Network Intrusion Detection System (NIDS) that keeps monitoring the traffic in real-time and notifies about any abnormal traffic and potential attacks [1]. The complexity and difficulty of the intrusion-detecting challenge are increasing noticeably, especially in new networks like 5G networks that have dynamic architectures, where the number of devices connected to the network increases rapidly every day, and the traffic of data grows exponentially [2]. This growing complexity increases the number of vulnerabilities and possible threats because of the diversity, hugeness, and overabundance of applications used in these networks [3]. Therefore, securing 5G networks is a crucial matter [3]. The

rarity of 5G environments labelled data makes the challenges more difficult [2]. An accuracy of 99.4% [3], and 99.6% [4] look impressive, but in the field of intrusion detection, especially in 5G networks, it is still not enough because this means that a small number of possible cases of false positives or false negatives still exist and cause serious issues for an IDS and have dangerous consequences. **Use Case:** suppose we have a smart grid enabled by a 5G network. This grid manages energy distribution in real-time. It has devices and sensors connected to it internally to make sure that the energy is managed efficiently and to keep the grid stable. In case of a false negative, there is an intrusion missed without being detected. For instance, suppose a distributed denial of service attack over the grid's control system is missed; widespread blackouts could occur, which is very dangerous. It could lead to huge financial loss, and human lives may be threatened.

Having an IDS with 99% accuracy still leads to missing 1% of attacks, which could be catastrophic. In case of a false positive, some unnecessary decisions will be made, like disabling or isolating a part of the grid, leading to downtime in non-compromised parts and suspending normal life there. The intrusion detection system (IDS) is a vital method used as a security tool to protect data assets or network connections and ensure their safety. Intrusion Detection Systems (IDS) can be classified as Host-Based (HIDS), which monitor and protect the device on which they are installed, or Network-Based (NIDS), which inspect network traffic at the perimeter of a local network. NIDS can be signature-based, relying on a database of known attack patterns—effective only for known threats—or anomaly-based, which models normal traffic behaviour and flags deviations as potential intrusions, allowing the detection of novel attacks. Nowadays, anomaly-based approaches of NIDS (that use DL or ML techniques) are more popular than signature-based approaches as they can detect new attacks.

Machine Learning (ML) and Deep Learning (DL) techniques are widely used in detecting intrusions, and various datasets have been created [5], [6], [7]. However, ML-based IDS and DL-based IDS usually suffer from a high false positive rate [8], which is bad for IDS generally, as it means that the system will give you alerts even when there is no attack, making the security team confused, and maybe some dangerous and critical alerts will be ignored. ML techniques are frequently used in classification problems, especially detecting attacks. However, usually, one technique performs very well in detecting some types of attacks and very badly in detecting others [9]. Ensemble learning depends on multiple weak base learners to make decisions, and each base learner or classifier is trained and tested to detect a particular class or a particular group of classes, and the overall prediction is made by combining the results using some blending technique [10]. Stacking is one of the ensemble learning techniques that train multiple base models on subsets of the training set; those subsets are taken randomly from the training set with replacement. The outputs of the base models will be used as input for the meta-model, which will make the final decision. Ensemble learning models like stacking are simpler than DL approaches like CNNs and RNNs in terms of architecture. In addition, they need less computational power and resources compared to DL models. They need longer training time, but in prediction, they are faster than DL approaches. Also, the stacking model is definitely easier to interpret. Compared to other ensemble models used for IDS, stacking models provide better performance because:

- Stacking models can use a diverse set of base models; each one can learn different patterns. The meta-model then learns the best way to combine the predictions of those models, enhancing accuracy and reducing missed attacks.
- Stacking model can detect non-linear patterns, as it could have linear and non-linear base models.

- The stacking model mitigates individual model biases by using a meta-model that learns to assign weights to the base models' predictions before combining them. It combines their strengths and corrects false predictions.
- The stacking model uses base model predictions as features for training the meta-model, enriching data representation and improving class distinction.
- Stacking models are more flexible and customizable than bagging and boosting models, as they allow varied combinations of base and meta-models to optimise performance.
- Bagging models reduce only the variance by using multiple models and training them in parallel, but all the models are the same. Meanwhile, a diverse set of models is used in stacking, which reduces the bias, too.
- Compared to the stacking models, boosting models are more exposed to overfitting as they focus on reducing bias by training models sequentially, iteratively and correcting errors of previous models, while the stacking model doesn't have this problem as it learns the optimal combination of the base models' predictions.
- The stacking model is better at handling the class imbalance problem, as each base learner handles the data imbalance in a different way; this means that even rare attacks can be detected. The meta-model also gives more weight to the prediction of the model that is better at detecting the rare attacks.

Compared to hybrid models (where 2 or more models are combined sequentially), stacking models are preferable as:

- Hybrid models lack flexibility, as the algorithms they use are already defined and don't change dynamically according to the data, while the stacking model has this flexibility as the meta-learner learns to find the optimal combination of predictions, and this optimal combination differs according to the input data.
- In specific scenarios, the stacking model learns which base models are performing better and adjusts according to that, which improves the performance dynamically, but hybrid models are fixed and can't do this.

For 5G environments, where the traffic is changing dynamically, and attacks can be more sophisticated and difficult to detect, and because of the previously mentioned reasons, the stacking model can be very efficient in 5G networks and other types of traditional networks too, especially, if the stacking architecture is well selected and studied thoroughly.

One of the issues we face in a dataset when trying to build a classification model is the imbalance. Some techniques could be used to bypass this issue, such as oversampling and undersampling techniques [11]. In oversampling, a duplication for samples from the minority class is done until the number of samples in all classes is equal, while in undersampling, samples belonging to majority classes are reduced at random so that there is an equal number of samples in all classes and the dataset is balanced [12]. Synthetic Minority Over-sampling Technique known as (SMOTE) is one of the most

common techniques used to solve the imbalance issue. it was in 2002 [13]. This technique generates new samples belonging to the minority class by interpolating between the original minority class samples [13]. The SMOTE technique is chosen because it gives better results compared to results when no technique for balancing the data was used and when other resampling techniques were used, as per our experiments. The problem that this article focuses on is the difficulty of reducing false positive and false negative rates at the same time, and the lack of dynamism in the existing approaches used for detecting intrusions. This article proposes a deep-stacking-based approach for detecting intrusions (DSEM-NIDS). This approach bypasses the drawbacks of each base model used in the stacking architecture and achieves high performance on different NIDS datasets (a 5G network dataset, a hybrid network dataset, an IoT network dataset and a classic dataset usually used as a benchmark) in terms of accuracy, precision, recall, and f1-score. It also reduces false positive rate (FPR) and false negative rate (FNR), lowers prediction time, and provides flexibility and dynamism when it is trained on different datasets. Our contribution is summarised as follows:

- Focusing on handling both overfitting and underfitting issues simultaneously.
- The novel structure of the model used for detecting intrusions, as per our knowledge and our studying of the state-of-the-art research in this field, this deep stacking model is new.
- The proposed model has the ability to learn dynamically according to the input data.
- Our model guarantees a high detection rate and a low false positive rate, which has always been a challenge in this domain, and at the same time, it requires fewer resources than resources required when using DL models.
- The proposed model has good scalability in terms of accuracy.

The rest of the article is organised as follows: a detailed “Literature Review” in Section II, “Methodology” in Section III, where the datasets used are introduced and the model is explained. In Section IV, “The Experiments” that are conducted to get the final results are explained thoroughly. The results are shown and discussed in Sections V and VI, and finally, we conclude the whole work in Section VII.

II. LITERATURE REVIEW

Denning [14] published a statistical anomaly detection model for intrusion detection. Since then, various IDS datasets and different approaches have been suggested. During the time after that, using ML techniques for building intrusion detection systems become a very interesting area of research. When talking about using ML in detecting intrusions, we are dealing with a classification problem where the ML model (which represents the IDS) acts like a classifier trained on a labelled training set. Some researchers realised that a single machine-learning model could suffer from either high bias or high variance; in other words, it could be either unable to learn the patterns (underfitting problem) or learn the patterns

too well (overfitting problem). Therefore, there was a move to treat this by using ensemble learning. Ensemble learning depends on multiple weak base learners to make decisions, and each base learner or classifier is trained and tested to detect a particular class or a particular group of classes, and the overall prediction is made by blending or combining the results using some blending technique [10]. The authors of [15] proposed a bagging ensemble method where REPTree base classifier is used to detect malicious packets. The results obtained show that the proposed model achieves the highest accuracy and that less time was needed to build the model. It has also been noticed that this model has a lower false positive rate compared to other models. In [9], an adaptive ensemble learning model is proposed with the goal of leveraging the advantages of various algorithms. The NSL-KDD dataset is used as a benchmark. This adaptive model, with multiple decision trees and voting techniques, was found to be better at detecting attacks than other techniques in previous research. In [16], the authors proposed ELNIDS, which is a network intrusion detection system based on ensemble learning, for detecting routing intrusions against the routing protocol IPV6 in low-power networks. Four ensemble learning models were used on trees: boosting, bagging, subspace discrimination and RUSBoosting. For evaluation, the RPL-NIDDS17 dataset was used. The boosting ensemble model was observed to give the best accuracy, and the RUSBoosted trees ensemble gave the best ROC. The authors of [17] proposed two DL-based models that mainly use CNNs to detect network intrusions and used two approaches of preprocessing, achieving high performance compared to the others’ works. They compared their models’ performances to the performance of other models; some of them are ensemble learning models like bagging. The article [18] presented an IDS based on ensemble learning, where voting was used to give the final prediction of three base learners, logistic regression, decision tree and naive Bayes. The model evaluation was done by using the CICIDS2017 dataset. This model outperforms state-of-the-art techniques in terms of accuracy in both binary and multi-class classification. The authors of [19] suggested an IDS based on a stacking ensemble learning model which combines the performance of random forest and gradient descent, and they found that the performance of the stacking model is better than bagging and boosting models, and also comparing to other ML techniques, the stacking model scores higher accuracy. In [20], an ensemble learning-based model was suggested to detect botnet intrusions; its name is ELBA-IoT. The N-BaIoT dataset was used as a benchmark, and it was found that the ELBA model outperforms other state-of-the-art works. One of the models that were used to evaluate the new UNR-IDD dataset that was proposed in [11] was the bagging model. This model scored one of the best performances on this dataset. Another stacking-based IDS was proposed in [21]; base learners were selected to be lightGBM and naive Bayes, and the meta-model selected is TPE-LightGBM. This model improves the performance and reduces the overhead of training and prediction time compared to the other basic techniques. In [22],

TABLE 1. Comparison of Ensemble Methods Used in the Related Works

Work	Year	Ensemble Method	Dataset	Drawbacks
[15]	2015	Bagging for REPTree base classifier	NSL-KDD	only one dataset is used to test the model/ the dataset is outdated/ the model categorises the packets into only 3 classes/ only accuracy and FPR are used as performance metrics/ there is no comparison with others' work.
[9]	2019	Adaptive Voting	NSL-KDD	only one dataset is used to test the model/ which is outdated/ only accuracy is used as a performance metric.
[16]	2019	Voting	RPL-NIDS17	only one dataset is used to test the model/ binary classification only/ there is no comparison with others' works.
[17]	2021	Bagging	NSL-KDD, UNSW-NB15	low accuracy
[18]	2022	Voting	CICIDS2017	only one dataset is used to test the model/ there is no comparison with others' works.
[19]	2022	stacking	NSL-KDD	only one dataset is used to test the model/ the dataset is outdated/ there is no comparison with others' works/ only accuracy is used as a performance metric.
[20]	2022	ELBA-IoT (voting)	N-BaIoT	Only one dataset is used
[11]	2022	BC (Bagging Classifier)	UNR-IDD	didn't compare the performance of BC on other datasets
[21]	2023	Stacking	N-BaIoT, UNSW-NB15	only binary classification
[22]	2023	Stacking	ToN-IoT, CICIDS2017, SWaT	only 4 parameters are used for evaluation
[4]	2024	Voting, Bagging	5G-NIDD	only one dataset is used/ there is no comparison with others' works
Our work	2024	Deep stacking model	5G-NIDD, UNR-IDD, N-BaIoT, NSL-KDD	-

we can find a proposal for a new approach of stacking-based IDS in IoT. This model combines the performances of CNN, DNN, LSTM and MLP and uses an integrated neural network as a meta-learner. CICIDS2017 and ToN-IoT datasets were used as a binary and multi-class classification benchmark. The proposed model scores the best results compared to previous works and also keeps the rates of false positives and false negatives low. The authors of [4] compared 13 ML-based IDS models in an environment that is controlled and monitored accurately; they used the 5G-NIDD dataset as a benchmark. A comparison of ensemble methods used in the related works is shown in Table 1. The authors of [23] enhanced the performance of AdaBoost-based IDS on the CICIDS2017 dataset by using SMOTE, PCA, and Ensemble feature selection. They found that the model with SMOTE outperforms other models and also the same model without SMOTE. In [24], SMOTE is used to increase the number of samples in the minority classes, and this improves the performance of the combination of sequential classifiers, that were opposed to improving the false detection. Using SMOTE with the proposed sequential classifiers increased the accuracy and sensitivity and enabled the model to detect more malicious cases. Also, IN [25], SMOTE is used to solve the problem of class imbalance and enhance the performance of two IDS models on both CICIDS2017 and NSL-KDD datasets. In addition to the studies discussed above, other recent research has explored diverse approaches to intrusion detection. Authors of [26] proposed an IDS combining neural networks with the Firefly Algorithm.

The article [27] introduced a self-adaptive IDS to enhance UAV-to-UAV communication security. The work [28] developed a Big Data-aware IDS using deep learning to manage high-volume network traffic. These works provide valuable perspectives on the evolution of intelligent intrusion detection techniques.

III. METHODOLOGY

A. DATASETS

1) 5G-NIDD DATASET

It was created by capturing the traffic of a 5G network functioning while several attack scenarios are conducted, like DoS attacks and Port scans, and also it contains benign traffic from normal users [3], [4], [29], [30]. 5G-NIDD dataset is labelled and can be used for binary and multi-class classification. It has 1,215,890 samples. In this article, we use it for multi-class classification, and our model can detect all its classes (Normal, UDPFlood, HTTPFlood, SlowrateDoS, Port TCPConnectScan, SYNScan, UDPScan, SYNflood, ICMPFlood).

2) UNR-IDD DATASET

It was revealed in 2022 by Nevada-Reno University. It was collected using a Software-Defined Network simulation environment [11], [31], [32]. The dataset contains 32 features and two columns for labelling; one is for binary classification, and the other is for multi-class classification. Nine features are collected using port statistics, another nine features are collected using delta port statistics, and 12 features are collected from flow entry and flow table statistics [11]. UNR-IDD dataset has 37412 samples and five types of attacks (in case of multi-class classification), as we can find the following labels in the column "label": TCP-SYN flood, Port scan, Flow table overflow, Blackhole, Diversion, and Normal [11]. The proposed model can detect all of the attack types.

3) N-BAIOT DATASET

Because of the shortage of publicly available botnet datasets, this dataset was created to gather real traffic from 9 IoT devices infected by Mirai and BASHLITE. In total, it has 7062606 records with 10 classes of attacks and 1 benign class, and 115 features [33], [34], [35], [36].

4) NSL-KDD

It is an improved version of the KDD cup'99 dataset [5], [37], and it is very much used as a benchmark in research related to intrusion detection [38], [39], [40]. NSL-KDD solved the problem of redundant records in KDD Cup'99 dataset. It has 5 labels: Normal, DoS, U2R, R2L, and Prob, and it has 41 features.

B. OUR MODEL

In this article, we propose an ensemble learning model. It is a nested stacking model consisting of 8 base learners, one of them is another stacking model itself. The goal of any stacking model is to use the meta-learner to learn the best way of

combining the predictions of the learners in the first level. The following base learners are used:

- *Logistic Regression (LR)*: A statistical classification method that models feature correlations and their common effects to effectively separate classes, making it suitable for intrusion detection [41].
- *k Nearest Neighbours (KNN)*: Classifies a datapoint by measuring its distance to training samples and assigning it the class of the majority among its k nearest neighbours [42].
- *Decision Tree (DTC)*: A tree-like model where internal nodes test features, branches represent outcomes, and leaves indicate class labels. Classification is done by following a path from root to leaf based on feature values [43].
- *SVM*: A margin-based classifier that finds an optimal hyperplane to separate classes while minimising structural risk. It generalises well and helps reduce overfitting [44].
- *Gaussian Naive Bayes (GNB)*: Gaussian Naive Bayes is a numeric variant of the Naive Bayes classifier, which assumes feature independence. Despite its simplicity, it is efficient for intrusion detection [45] [46].
- *Random Forest (RF)*: Random Forest is an ensemble of independent decision trees built on bootstrap samples. It predicts by majority voting across trees [47] [48].
- *XGBoost (XGB)*: eXtreme Gradient Boosting builds a sequential ensemble of decision trees. Each tree corrects the errors of the previous one until convergence is reached [49].
- *Stacking Model*: An ensemble learning technique that combines the predictions of multiple base learners using a meta-learner. The base models form the first layer, while the meta-model in the second layer learns how to best integrate their outputs [50].

The stacking model consists of two layers. The first layer contains the base learners, who parallelly predict the output from the given input. The second layer contains a single meta-learner to combine the results of the first layer. The stacking model used in this article has the following models: LR, DT, KNN, GNB, SVC, RF, and XGB as first-layer base learners and XGB as second-layer meta-learner. The overall model proposed here is a deep-stacking model, which means one of the complete stacking models is added as one of the base learners in the first layer, as shown in Fig. 1. The Algorithm 1 explains step by step how the model works.

Let $\{(x_i, y_i)\}_{i=1}^N$ denote the training dataset, where $x_i \in \mathbb{R}^d$ and $y_i \in \{1, 2, \dots, C\}$. The proposed DESM-NIDS framework consists of 7 base learners $\{M_i\}_{i=1}^7$, each of which maps an input x to a prediction $\hat{y}_i = M_i(x)$.

The outputs of the base learners form a new feature vector $\hat{y}^{(1)} \in \mathbb{R}^7$, which is input to a meta-learner M_L . The first-level prediction is thus: $\hat{y}_1 = M_L(\hat{y}^{(1)})$. In the second stage, the base learners are reapplied to obtain $\hat{y}^{(2)}$, and the final input to the meta-learner is $[\hat{y}^{(2)}, \hat{y}_1]$, which yields the final prediction $\hat{y}_2$. The learning process optimizes a cross-entropy loss at each stage.

Algorithm 1: The DESM-NIDS Algorithm.

```

1: Input Data:  $D(X_n, Y_n)$ 
2: Base Learners:
    $M_1 = LR; M_2 = DT; M_3 = KNN; M_4 =$ 
    $GNB; M_5 = SVM; M_6 = RF; M_7 = XBG$ 
3: Base Learners Predictions:  $\hat{Y}_{1M_i}$ 
4: Meta Learner:  $M_L = XGB$ 
5: First stage's Final Prediction:  $\hat{Y}_1$ 
6: Second Stages's Final Prediction:  $\hat{Y}_2$ 

7: Step 1: Classify  $D$  With The Base Learners from  $M_1$ 
   to  $M_7$ 
8: for  $i = 1$  to 7 do
9:    $\hat{Y}_{1M_i} = M_i\{D\}$ 
10: end for

11: Step 2: Concatenate Data:  $D' =$ 
    $\{(\hat{Y}_{1M_1}, \hat{Y}_{1M_2}, \hat{Y}_{1M_3}, \hat{Y}_{1M_4}, \hat{Y}_{1M_5}, \hat{Y}_{1M_6}, \hat{Y}_{1M_7}), Y_n\}$ 

12: Step 3: Train The Meta Learner  $M_L$ :
13:  $\hat{Y}_1 = M_L\{D'\}$ 
14: return  $\hat{Y}_1$ 

15: Step 4: Second Round of Training:
16: for  $i = 1$  to 7 do
17:    $\hat{Y}_{1M_i} = M_i\{D\}$ 
18: end for

19: Step 5: Concatenate Data:  $D'' =$ 
    $\{(\hat{Y}_{1M_1}, \hat{Y}_{1M_2}, \hat{Y}_{1M_3}, \hat{Y}_{1M_4}, \hat{Y}_{1M_5}, \hat{Y}_{1M_6}, \hat{Y}_{1M_7}, \hat{Y}_1), Y_n\}$ 

20: Step 6: Train Meta Learner on The New Data:
21:  $\hat{Y}_2 = M_L\{D''\}$ 
22: return  $\hat{Y}_2$ 

```

The meta-learner is trained via gradient boosting using **XGBoost**. Let:

- $D = \{(x_i, y_i)\}_{i=1}^N$ be the training dataset,
- $x_i \in \mathbb{R}^d$ be the input feature vector,
- $y_i \in \{1, 2, \dots, C\}$ be the class label.

XGBoost builds an ensemble of K decision trees, where each tree f_k predicts a score for the input x_i . The final model prediction is: $\hat{y}_i = \sum_{k=1}^K f_k(x_i)$

The corresponding cross-entropy loss is:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^C \mathbf{1}(y_i = c) \log(\hat{p}_{i,c})$$

where:

- $\hat{p}_{i,c}$ is the predicted probability (after softmax) that sample i belongs to class c ,
- $\mathbf{1}(y_i = c)$ is the indicator function that is 1 if class c is the true label.

Optimisation: XGBoost improves the model in stages by adding trees that correct the errors of the previous trees using gradient boosting.

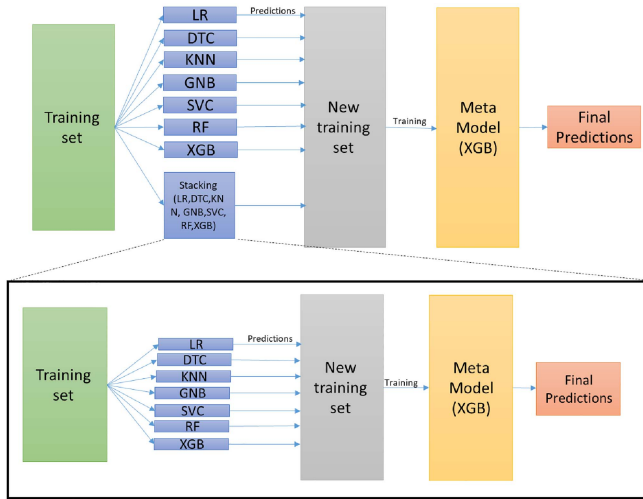


FIGURE 1. The proposed structure: the blue boxes represent the first layer of base models, and the second layer of the stacking model is in yellow.

This model was chosen for its competitive performance across datasets, simpler architecture, and lower computational requirements compared to deep learning. It is also more flexible, customizable, less prone to overfitting, and better at handling class imbalance than other ensemble models (as detailed in the introduction). Compared to hybrid models, it offers greater flexibility and dynamism. As shown in Table 1, our model overcomes common drawbacks by testing on four diverse and recent datasets, performing multiclass classification, benchmarking against other works, using multiple metrics, and conducting scalability tests.

IV. EXPERIMENTS

Preprocessing. *5G-NIDD*: Features that contain a large fraction of null values were removed. The categorical features were converted to numerical values using *get_dummies* function from *pandas* package. Using ANOVA, we selected the best 22 features (ANOVA is selected as a feature selection technique because we have a numerical input and categorical output, and our problem is a classification problem [51]), and after that, we normalised the data using *StandardScaler*.

UNR-IDD: First, we checked the dataset for any missing or infinite values, and we did not find any. The feature (*is_valid*) was removed because it has the same value (True) for all data points. Other features like *switch-id* and *port number* are removed. We transformed the values of the categorical features into numerical values using *LabelEncoder* function, and we did the same to the values of the column (*Label*), as this column indicates the classes that we will use to classify the traffic. The most effective 14 features are selected using the ANOVA technique. ANOVA is a feature selection method that uses the ratio between the variances to rank the features [52]. ANOVA is our choice because we have a numerical input and a categorical output, and our problem is a classification problem [51]. we normalized the data using *StandardScaler*.

N-BaIoT: A subset of the dataset is used for training; the following files are used: *5.benign*, *5.gafgyt.combo*, *5.gafgyt.junk*, *5.gafgyt.scan*, *5.gafgyt.tcp*, *5.gafgyt.udp*, *5.mirai.ack*, *5.mirai.scan*, *5.mirai.syn*, *5.mirai.udp*, *5.mirai.udpplain*. The selected files are concatenated in one file, and then *LabelEncoder* is used to encode the categorical values in the column 'type' that represents the label of each record. The values in each column are standardised by calculating each column's mean and standard deviation, then subtracting the column's mean from each value in the column and dividing it by its standard deviation, which gives the column a mean of 0 and a standard deviation of 1.

NSL-KDD: *KDDTrain+* file is used for training, and *KDDTest+* file for testing. First, we concatenate the two files; then, we use *OneHotEncoder* to transform nominal features into numerical features, and the numerical values (that were originally numerical) are scaled using *MinMaxScaler*. The 'attack' column values are mapped to 5 classes, 4 classes of attacks and 1 class for benign. After that, the obtained file is split again into train and test of the same size as the *KDDTrain+* and *KDDTest+*.

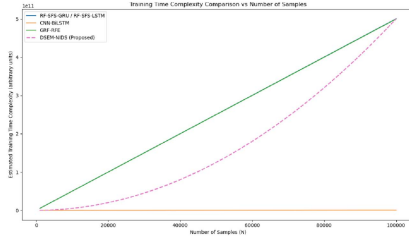
Data Splitting: for all datasets, we took 80% of data for training and 20% for testing. For each dataset and training experiment, 5-fold cross validation was used.

Balancing dataset: To balance the UNR-IDD dataset, various resampling methods were tested, such as random oversampling, borderline SMOTE, SVM SMOTE, *adasyn*, SMOTE ENN, SMOTE-Tomek, and SMOTE. SMOTE was selected as the best-performing method for resampling.

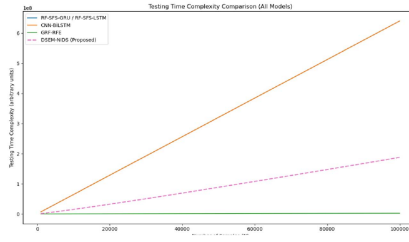
For all experiments, the following configuration is used: a workstation with 13th Generation, Core i9 processor, 24 cores, and 128 GB RAM. As we mentioned before, ensemble learning aims to combine the predictions of multiple weak learners to get better performance. Weak learners could be any learner who suffers from either high variance or high bias; in other words, they could suffer from either overfitting or underfitting. Compared to DL methods, ensemble learning is still simpler, needs less computational power, less prediction time, and is indeed easier to interpret.

V. RESULTS

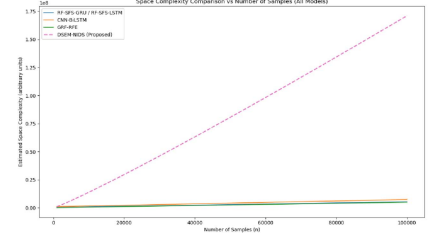
Accuracy: Tables 3–8 demonstrate that DSEM-NIDS consistently outperforms all baseline and state-of-the-art models in terms of accuracy, precision, recall, and F1-score across four datasets. Table 3 compares our model's performance on the 5G-NIDD dataset per class with models from [3], showing superior results. Table 4 reinforces this by comparing our model's overall performance and execution time against base learners and recent studies—again highlighting DSEM-NIDS's advantage. Tables 5 and 6 show that on the UNR-IDD dataset, our model outperforms models from [11], [31], and [32], except for slightly lower precision in the overflow class—still with a better F1-score. In Tables 7 and 8, our model achieves the best results on the N-BaIoT and NSL-KDD datasets compared to both base models and recent literature. Finally, Table 9 confirms that DSEM-NIDS



(a) DSEM-NIDS training time complexity vs. number of samples

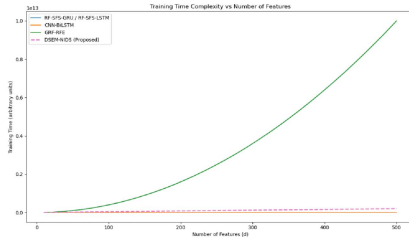


(b) DSEM-NIDS testing time complexity vs. number of samples

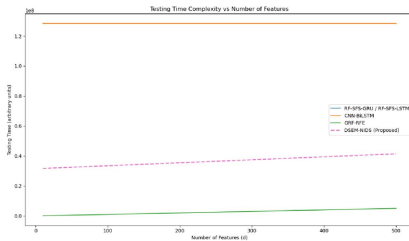


(c) DSEM-NIDS space complexity vs. number of samples

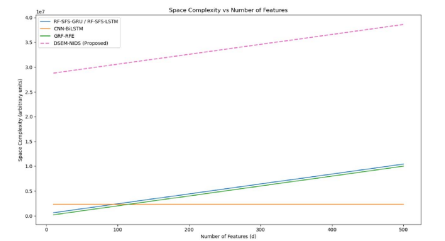
FIGURE 2. Complexity changes vs. number of samples n compared to complexity changes of models used in [62], [63], [64], [65].



(a) DSEM-NIDS training time complexity vs. number of features



(b) DSEM-NIDS testing time complexity vs. number of features



(c) DSEM-NIDS space complexity vs. number of features

FIGURE 3. Complexity changes vs. number of features d compared to complexity changes of models used in [62], [63], [64], [65].

TABLE 2. Time and Space Complexity

Component	Training time complexity	Testing time complexity	Space complexity
SMOTE	$O(n_{minority} \cdot k \cdot d)$ [13]	-	$O(n \cdot d)$ [13]
LR	$O(n \cdot d)$ [53]	$O(d)$ [53]	$O(d)$ [53]
DTC	$O(n \cdot d \cdot \log n)$ [54]	$O(n \cdot d)$ [54]	$O(n \cdot d)$ [54]
KNN	$O(1)$ [55]	$O(n \cdot d)$ [55]	$O(n \cdot d)$ [55]
GNB	$O(n \cdot d)$ [56]	$O(d)$ [56]	$O(d)$ [56]
SVC	$O(n^3 \cdot d)$ [57]	$O(n_{support} \cdot d)$ [57]	$O(n \cdot d)$ [57]
RF	$O(t \cdot n \cdot \log n)$ [58] [59]	$O(t \cdot depth)$ [58]	$O(t \cdot depth)$ [58] [59]
XGB	$O(K \cdot n \cdot \log n)$ [60]	$O(K \cdot d)$ [60]	$O(K \cdot depth)$ [60]
Stacking XGB	$O(K' \cdot n \cdot \log n)$ [60] [61]	$O(K' \cdot B)$ [60] [61]	$O(K' \cdot depth)$ [60] [61]
DSEM-NIDS	$O(n_{minority} \cdot k \cdot d) + \sum_{i=1}^B T_{Base_i}(n, d) + O(K \cdot n \cdot \log n)$	$\sum_{i=1}^B P_i(d) + O(K \cdot d)$	$O(n \cdot d) + \sum_{i=1}^B M_i + O(K' \cdot depth)$

Note: Complexities are either directly stated or derived based on algorithm descriptions from the cited sources.

maintains low variance and acceptable bias across datasets, indicating robustness and good generalizability.

Scalability: Figs. 4 and 5 show how the accuracy changes when the training set size changes from 10% to 90% on 5G-NIDD and UNR-IDD datasets when running DSEM-NIDS and the other base models.

Impact of feature selection: Figs. 6 and 7 show how the accuracy changes when the features number increases on 5G-NIDD and UNR-IDD datasets when running DSEM-NIDS and the other base models.

Time and Space Complexity: Using Big O notation, let n be the sample count, d the feature count, t the number of trees, K the boosting rounds, k the nearest neighbors, B the base learners, P_i the per-sample time for each base learner, T_{Base} the total base model training time, and M_i the memory per

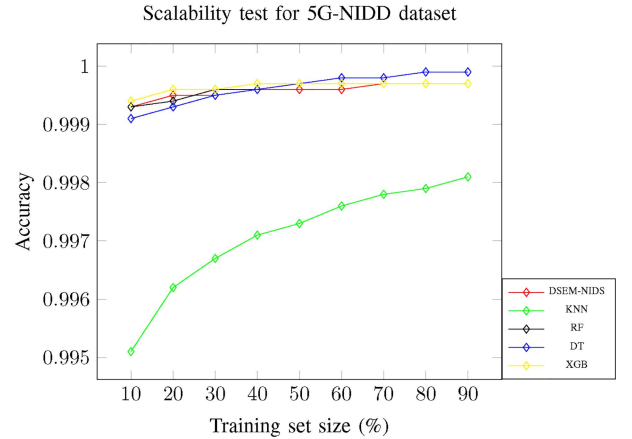


FIGURE 4. Scalability test results for the 5G-NIDD dataset.

model. Table 2 presents the total training time, testing time, and space complexities of DSEM-NIDS. Fig. 2 shows how these complexities grow with n , and Fig. 3 shows their growth with d .

VI. DISCUSSION

A. PERFORMANCE

Accuracy measures the ratio of correctly classified samples. Based on the results, our model achieves the highest accuracy across four datasets, demonstrating its robustness and strong ability to distinguish between classes. **Precision** Precision reflects the model's ability to correctly identify positive instances (attacks), making it a key metric for intrusion

TABLE 3. Comparison Between Results of DSEM-NIDS on 5G-NIDD Dataset and Results of Other ML Techniques Used in the Original Paper of Dataset [3]

Metric	DSEM-NIDS										DT [3]					Random Forest [3]					KNN [3]					Naïve Bayes [3]					MLP [3]							
	Ben	HTTP	ICMP	SYNF	SYNS	Slow	TCP	UDPF	UDPS		Ben	HTTP	ICMP	SYNF	SYNS	Slow	TCP	UDPF	UDPS		Ben	HTTP	ICMP	SYNF	SYNS	Slow	TCP	UDPF	UDPS	Ben	HTTP	ICMP	SYNF	SYNS	Slow	TCP	UDPF	UDPS
Precision	0.9994	0.9992	0.9996	1.0000	1.0000	1.0000	0.9987	1.0000	1.0000		0.9931	0.9924	0.9931	0.9883	0.9871	0.9895	0.9885	0.9878	0.9882		0.9971	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970
Recall	0.9992	0.9998	0.9998	1.0000	1.0000	1.0000	0.9997	1.0000	1.0000		0.9886	0.9879	0.9879	0.9878	0.9878	0.9895	0.9885	0.9878	0.9882		0.9982	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972	0.9972
F1-score	0.9992	0.9995	0.9997	1.0000	1.0000	1.0000	0.9997	1.0000	1.0000		0.9931	0.9924	0.9931	0.9883	0.9871	0.9895	0.9885	0.9878	0.9882		0.9971	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970	0.9970
Total Acc.	0.9995										0.9917					0.9948					0.9948					0.4099					0.9950							

Note: Ben: Benign, HTTP: HTTP Flood, ICMP: ICMP Flood, SYNF: SYN Flood, SYNS: SYN Scan, Slow: Slowrate DoS, TCP: TCP Connect Scan, UDPF: UDP Flood, UDPS: UDP Scan

TABLE 4. Results of DSEM-NIDS Performance on 5G-NIDD Dataset Compared to the Results of State-of-The-Art Models

Model	Accuracy	Precision	Recall	F1	ET (sec)
DT	0.9993	0.9991	0.9994	0.9993	1
RF	0.9992	0.999	0.9994	0.9992	1.01
GNB	0.7495	0.9303	0.9146	0.9015	1.001
SVM	0.9959	0.9921	0.9948	0.9935	1.004
KNN	0.9971	0.9935	0.9968	0.996	0.012
LR	0.9849	0.9803	0.9809	0.9805	1
XGB	0.9996	0.9993	0.9997	0.9995	1.01
DSEM-NIDS	0.9997	0.9997	0.9997	0.9997	0.11
Naive Bayes [4]	0.942	0.948	0.90	0.924	0.014
LDA [4]	0.954	0.993	0.889	0.938	0.049
DT [4]	0.816	0.681	0.999	0.81	0.101
KNN [4]	0.996	0.997	0.993	0.995	0.17
LR [4]	0.96	0.995	0.903	0.947	0.193
Bagging [4]	0.816	0.681	0.999	0.81	0.732
Extra Trees [4]	0.91	0.815	0.997	0.897	1.101
AdaBoost [4]	0.554	0.415	0.335	0.371	1.815
Random Forest [4]	0.983	0.96	1	0.979	3.387
Gradient Boosting [4]	0.853	0.728	0.999	0.842	7.233
SVM [4]	0.968	0.997	0.922	0.958	15.183
NN [4]	0.98	0.998	0.95	0.974	37.56
Voting Classifier [4]	0.988	0.999	0.97	0.984	0.732
RNN [29]	0.9748	0.9784	0.9573	0.9678	-
CNN [29]	0.9756	0.9925	0.9454	0.9683	-
FNN [29]	0.9753	0.9892	0.9478	0.968	-
GRU [29]	0.9588	0.9734	0.9206	0.9463	-
LSTM [29]	0.9801	0.9828	0.9664	0.9746	-
BiLSTM [29]	0.9809	0.9865	0.9648	0.9755	-
CNN [30]	0.941	0.971	0.9156	0.9425	-
ResNet [30]	0.9567	0.9916	0.9276	0.9584	-
Inception [30]	0.9723	0.9808	0.9647	0.9727	-
BiLSTM [30]	0.9874	0.9962	0.9790	0.9875	-

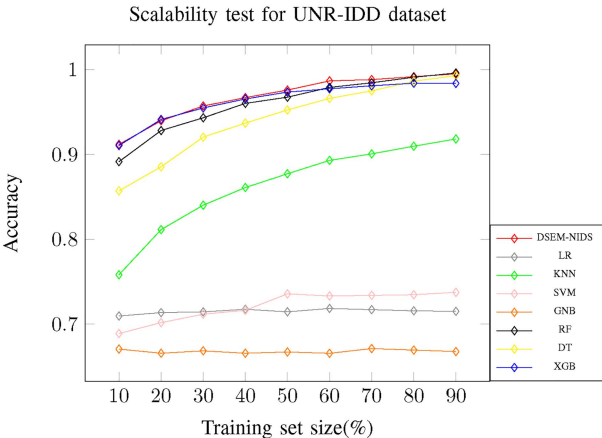


FIGURE 5. Scalability test results for the UNR-IDD dataset.

TABLE 5. Results of DSEM-NIDS on UNR-IDD Dataset Compared to the Results of Other Models in the Literature

Model	Accuracy	Precision	Recall	F1-score
DT	0.9246	0.901	0.9197	0.9094
RF	0.9571	0.9483	0.9637	0.9554
GNB	0.6687	0.6553	0.6357	0.9198
SVM	0.7309	0.7401	0.7775	0.7094
KNN	0.8643	0.8276	0.8708	0.8417
LR	0.7177	0.7078	0.7414	0.6936
XGB	0.964	0.9692	0.9648	0.967
DSEM-NIDS	0.9712	0.9718	0.9737	0.9727
SVM [11]	-	0.89	0.79	0.81
MLP [11]	-	0.59	0.54	0.54
RF [11]	-	0.96	0.92	0.94
BC [11]	-	0.95	0.93	0.94
KNC [11]	-	0.79	0.75	0.77
ABC [11]	-	0.69	0.59	0.55
XGBoost w feature reduction [31]	-	-	-	0.8979
XGBoost w/o feature reduction [31]	-	-	-	0.9058
optimized NN [32]	0.92	-	-	-

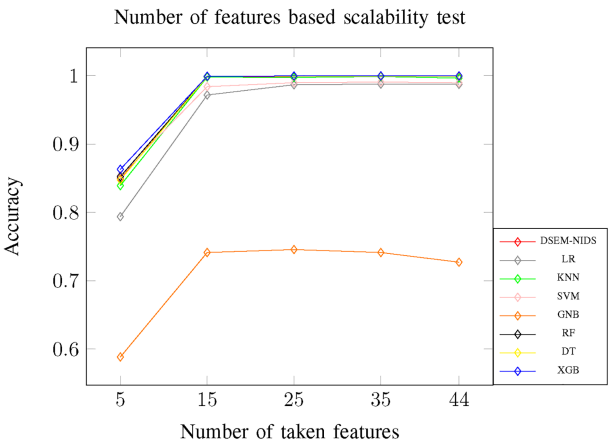


FIGURE 6. Change of accuracy of different models when the number of features taken from 5G-NIDD dataset increases.

detection. High precision indicates a low false positive rate, which is crucial in 5G and IoT environments. As shown in Table 3, our model consistently achieves higher precision across all classes in the 5G-NIDD dataset. Table 4 further confirms that our model's overall precision (99.97%) surpasses

TABLE 6. Comparison of Each Class Results When Using Our DSEM-NIDS on the UNR-IDD Dataset Vs. Results of RF Used in the Original paper [11] and Results of Optimized Neural Network Via Spider Monkey Algorithm [32]

	Blackhole			Diversion			Normal			Overflow			Portscan			TCP-SYN		
Parameter	Ours	RF [11]	NN [32]	Ours	RF [11]	NN [32]	Ours	RF [11]	NN [32]	Ours	RF [11]	NN [32]	Ours	RF [11]	NN [32]	Ours	RF [11]	NN [32]
Precision	0.9994	0.98	0.97	0.9973	0.99	0.98	1	1	1	0.9423	0.98	0.90	0.9431	0.91	0.84	0.9476	0.91	0.83
Recall	0.997	0.98	0.95	0.9973	0.97	0.97	1	1	1	0.956	0.76	0.95	0.9508	0.94	0.83	0.9402	0.92	0.83
f1score	0.9981	0.98	0.96	0.9973	0.98	0.98	1	1	1	0.9491	0.86	0.92	0.9469	0.92	0.84	0.9438	0.92	0.83

TABLE 7. Results of DSEM-NIDS on N-BaIoT Dataset Compared to the Results of Other Models

Model	Accuracy	Precision	Recall	F1-score
DT	0.8749	0.9241	0.8749	0.8332
RF	0.875	0.9371	0.875	0.8333
GNB	0.7914	0.755	0.7914	0.7388
SVM	0.8603	0.9231	0.8603	0.818
KNN	0.9992	0.999	0.9991	0.999
LR	0.8385	0.781	0.8385	0.7958
XGB	0.999957	0.999915	0.999937	0.999924
DSEM-NIDS	0.99998	0.999996	0.99998	0.99997
1D CNN [66]	0.996	0.998	0.996	0.997
BLSTM1 [67]	0.8638	-	-	-
ADA-IoT [20]	0.973	0.955	0.922	0.939
RUS-IoT [20]	0.977	0.969	0.957	0.963
BAG-IoT [20]	0.962	0.924	0.906	0.915
ELBA-IoT [20]	0.996	0.984	0.971	0.977

TABLE 8. Results of DSEM-NIDS on NSL-KDD Dataset Compared to the Results of Other Models

Model	Accuracy	Precision	Recall	F1-score
DT	0.7727	0.8011	0.7727	0.752
RF	0.7587	0.8176	0.7587	0.7158
GNB	0.4416	0.6218	0.4416	0.4609
SVM	0.7887	0.8243	0.7887	0.7723
KNN	0.7722	0.7995	0.7722	0.7723
LR	0.7965	0.8204	0.7965	0.7737
XGB	0.7951	0.8293	0.7951	0.7751
DSEM-NIDS	0.8005	0.8372	0.8005	0.7827
DT [68]	0.75	0.767	0.75	0.702
RF [68]	0.748	0.745	0.748	0.699
ET [68]	0.753	0.788	0.753	0.706
XGBoost [68]	0.746	0.793	0.746	0.701
NB [68]	0.368	0.39	0.368	0.277
KNN [68]	0.734	0.725	0.734	0.684
SVM [68]	0.794	0.732	0.794	0.743
Ensemble [68]	0.757	0.779	0.757	0.707
AdaBoost [69]	0.70	0.84	0.70	0.70
J48 [17]	0.7499	0.796	0.75	0.711
Bagging [17]	0.7483	0.783	0.748	0.716
Enhanced RF [70]	0.7874	-	78	-
Bagging for REPTree base classifiers [15]	0.8129	-	-	-

TABLE 9. Variance and Bias Trade-Off

Model	SG-NIDD		UNR-IDD		N-BaIoT		NSL-KDD	
	Variance	Bias	Variance	Bias	Variance	Bias	Variance	Bias
DSEM-NIDS	1.48e-8	-2.06e-5	1.05e-6	0.0004	8-10	-6.37e-5	1.83e-8	-0.27
DT	6.4e-10	-6.58e-5	3.64e-5	0.006	4.09e-9	-0.12	9.74e-8	-0.21
RF	1.83e-9	-0.0001	2.08e-6	0.022	3.86e-9	-0.13	2.43e-8	-0.39
GNB	3.49e-5	0.25	6.48e-6	0.39	3.66e-5	0.15	0.002	-0.004
SVM	3.8e-9	0.0005	1.52e-5	0.18	5.7e-8	-0.14	6.9e-8	-0.14
KNN	1.5e-8	0.001	1.7e-5	0.023	0.0003	0.004	1.01e-8	-0.26
LR	5.1e-8	0.006	1.94e-6	0.17	1.25e-6	0.11	3.73e-7	-0.19
XGB	8.73e-10	-2.47e-5	4.92e-6	0.015	7.97e-10	3.02e-5	1.3e-8	0.27

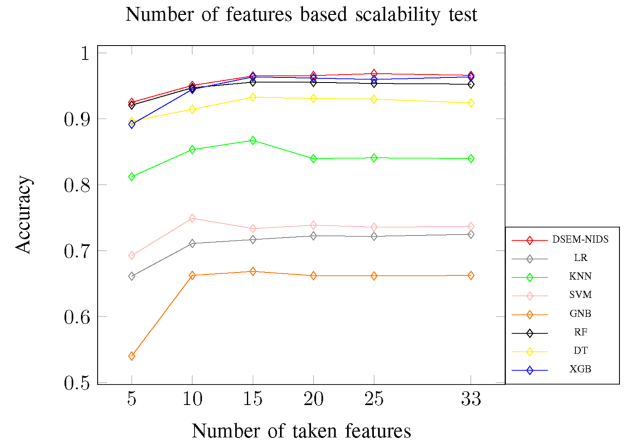


FIGURE 7. Change of accuracy of different models when the number of features taken from UNR-IDD dataset increases.

all other recent works and individual base models. Similarly, for UNR-IDD (Tables 5 and 6), our DSEM-NIDS model outperforms others, except for the overflow class, which we discuss in the F1-score section. On the N-BaIoT dataset (Table 7), our model also achieves the highest precision, even slightly outperforming XGB—an important difference in IoT contexts. Table 8 shows the same trend on NSL-KDD, further validating our model's reliability in minimising false positives. **Recall** measures the model's ability to correctly identify actual positive instances, indicating how well it minimises false negatives. On the 5G-NIDD dataset, our model achieves near-perfect recall for each class (Table 3), outperforming models used in [3]. The overall recall of 99.97% also exceeds that of recent works [4], [30], and [29], as well as the recall of individual base models. Similarly, our model achieves the highest recall on the N-BaIoT and NSL-KDD datasets, confirming its strong ability to detect true attacks and minimise false negatives. **F1-score** is a critical metric in intrusion detection, as it balances both false positives and false negatives. In IDS, missing real attacks (false negatives) or generating false alerts (false positives) are both undesirable. On 5G-NIDD, our model achieves the highest F1-score for each class compared to models in [3], and a total F1-score of 99.97%, outperforming recent works [4], [30], and [29], as well as individual base models. For the UNR-IDD dataset, our model achieves an F1-score of 97.27%, the best among results reported in [11], [31], and [32] (Table 5). While RF achieves the highest precision for the overflow class, our model delivers the best F1-score, which is more important

when both false positives and false negatives matter. Similarly, our model attains the highest F1-scores on N-BaIoT and NSL-KDD, reflecting its balanced and reliable performance across datasets. Table 9 presents the bias and variance values of our proposed DSEM-NIDS model compared to the base models used within it across four datasets. Bias reflects the deviation from the true values (underfitting), while variance indicates the sensitivity to changes in the training data (overfitting). A robust and scalable model maintains low variance and acceptable bias as the training data grows. As shown, DSEM-NIDS consistently exhibits low variance across all datasets, indicating minimal overfitting, and maintains acceptable bias, with only slight underfitting on NSL-KDD. In contrast, most base models exhibit either high variance or high bias. XGB is the closest competitor, demonstrating a strong balance; however, DSEM-NIDS maintains more consistent and predictable bias across datasets, making it more robust and generalizable.

B. COMPLEXITY

Fig. 2(a) illustrates a super-linear growth in training time as the dataset size increases, due to the use of SMOTE, computationally intensive base models (e.g., SVC and KNN), cross-validated stacking, and the meta-learner. This increase emphasises the importance of parallelisation for efficiency, although the one-time offline training makes the cost acceptable given the model's high predictive performance. Fig. 2(b) shows a linear relationship between testing time and dataset size, making the model suitable for real-time applications, as prediction time per sample remains stable. Fig. 2(c) also reveals a linear increase in space complexity with dataset size, which is acceptable for high-memory systems but may require optimisation in constrained environments. In Fig. 3(a), training time grows linearly with the number of features, due to increased computation per sample, reinforcing the value of feature selection. Similarly, Fig. 3(b) shows a linear increase in testing time with feature count, which impacts real-time performance. Fig. 3(c) confirms a linear growth in space complexity with more features, due to the need to store additional data.

C. SCALABILITY

Figs. 4 and 5 demonstrate how model accuracy varies with different training set sizes. On the 5G-NIDD dataset, DSEM-NIDS maintains high accuracy even with small training sets and shows stable performance as the dataset grows, indicating strong scalability and robustness—critical in 5G and IoT environments. Similarly, on the UNR-IDD dataset, DSEM-NIDS consistently outperforms base models across varying training sizes, confirming its reliability and generalizability. Figs. 6 and 7 show accuracy trends with different numbers of selected features using ANOVA. For both datasets, the model achieves optimal accuracy with at least 15 features; adding more features yields no significant gain. This highlights the model's scalability and efficiency when the feature count is 15 or more.

D. CHALLENGES

While the proposed DSEM-NIDS demonstrates promising predictive performance on benchmark datasets, several practical challenges must be addressed to facilitate its effective deployment in real-world 5G and IoT environments. These challenges are aligned with the specific characteristics of the deep stacking ensemble architecture and the nature of distributed, heterogeneous networks:

- *Inference Latency and Decision Delay*: Although DSEM-NIDS achieves high accuracy, the multi-layered ensemble structure increases inference time, which may hinder real-time detection in latency-sensitive 5G applications (e.g., autonomous vehicles or industrial automation). Future work will explore asynchronous inference pipelines and edge-offloading strategies using model partitioning to minimise detection delays.
- *Heterogeneity of IoT Traffic and Devices*: The performance of DSEM-NIDS may degrade when deployed across devices with varied traffic patterns, protocols, or behaviours not represented during training. To ensure robustness, domain adaptation techniques and online transfer learning will be investigated, enabling the model to dynamically adapt to unseen traffic patterns without retraining from scratch.
- *Resource-Aware Model Deployment*: DSEM-NIDS, being an ensemble of multiple learners including a nested stack, demands significant computational and memory resources. This challenge becomes critical at the edge, where such resources are limited. Future efforts will focus on building a lightweight version of DSEM-NIDS using neural architecture search (NAS), pruning, quantisation, and knowledge distillation, targeting deployment on microcontrollers and edge gateways.
- *Scalability in Federated or Multi-Tenant Settings*: In federated or multi-cloud IoT infrastructures, coordinating model updates and maintaining consistent detection logic across nodes introduces synchronisation and versioning overhead. We plan to incorporate scalable federated stacking strategies, ensuring consistent threat detection while preserving data locality and reducing communication costs.
- *Adversarial Robustness and Poisoning Attacks*: Stacking models can be sensitive to adversarial samples or poisoned training data that mislead individual learners. This concern is heightened in distributed IoT scenarios. Future research will integrate adversarial training and robust aggregation mechanisms into DSEM-NIDS to defend against model evasion and data poisoning threats.

VII. CONCLUSION

This article proposes a deep stacking model to detect intrusions. After conducting many experiments and trying many other models and different combinations, this model, with its nested structure, is chosen. It is a nested stacking model consisting of 8 base learners; one of them is another stacking

model itself, and it has a meta-learner that learns the best way to combine the predictions of the base learners, as their predictions are used to train the meta-learner. This makes the model flexible and adaptive to the input data. 5G-NIDD, UNR-IDD, NSL-KDD and N-BaIoT datasets are used to evaluate the model. The model assessment uses the following metrics: Accuracy, precision, recall, and F1-score. The results show that our proposed model outperforms other models and scores the best performance in terms of accuracy, precision, recall, and f1-score, reflecting the suggested model's robustness and its high ability to distinguish between different classes on different datasets. The proposed model has the best bias-variance trade-off compared to the base models on all four datasets and has very good scalability on both 5G-NIDD and UNR-IDD datasets from two perspectives: holding high accuracy while increasing training set size and when increasing the number of features used (at least 15 features are needed to get best results). The high performance of the proposed model on the four datasets and its good scalability on both the 5G-NIDD and UNR-IDD datasets give it a strong possibility to be generalised.

REFERENCES

- [1] X. Liu, T. Li, R. Zhang, D. Wu, Y. Liu, and Z. Yang, "A GAN and feature selection-based oversampling technique for intrusion detection," *Secur. Commun. Netw.*, vol. 2021, 2021, Art. no. 9947059.
- [2] B. Farzaneh, N. Shahriar, A. H. Al Muktafir, and M. S. Towhid, "DTL-IDS: Deep transfer learning-based intrusion detection system in 5G networks," in *Proc. 2023 19th Int. Conf. Netw. Service Manage.*, 2023, pp. 1–5.
- [3] S. Samarakoon et al., "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," 2022, *arXiv:2212.01298*.
- [4] M. A. Bouke and A. Abdullah, "An empirical assessment of ML models for 5G network intrusion detection: A data leakage-free approach," *e-Prime-Adv. Elect. Eng., Electron. Energy*, vol. 8, 2024, Art. no. 100590.
- [5] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD Cup 99 data set," in *Proc. 2009 IEEE Symp. Comput. Intell. Secur. Defense Appl.*, 2009, pp. 1–6.
- [6] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. 2015 Mil. Commun. Inf. Syst. Conf.*, 2015, pp. 1–6.
- [7] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," *ICISSp*, vol. 1, pp. 108–116, 2018.
- [8] J. Sinha and M. Manollas, "Efficient deep CNN-BiLSTM model for network intrusion detection," in *Proc. 2020 3rd Int. Conf. Artif. Intell. Pattern Recognit.*, 2020, pp. 223–231.
- [9] X. Gao, C. Shan, C. Hu, Z. Niu, and Z. Liu, "An adaptive ensemble machine learning model for intrusion detection," *IEEE Access*, vol. 7, pp. 82512–82521, 2019.
- [10] B. A. Tama and S. Lim, "Ensemble learning for intrusion detection systems: A systematic mapping study and cross-benchmark evaluation," *Comput. Sci. Rev.*, vol. 39, 2021, Art. no. 100357.
- [11] T. Das, O. A. Hamdan, R. Shukla, S. Sengupta, and E. Arslan, "UNR-IDD: Intrusion detection dataset using network port statistics," in *Proc. IEEE 20th Consum. Commun. Netw. Conf.*, 2023, pp. 497–500.
- [12] D. Gonzalez-Cuautle et al., "Synthetic minority oversampling technique for optimizing classification tasks in botnet and intrusion-detection-system datasets," *Appl. Sci.*, vol. 10, no. 3, 2020, Art. no. 794.
- [13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [14] D. Denning, "An intrusion-detection model," *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.
- [15] D. Gaikwad and R. C. Thool, "Intrusion detection system using bagging ensemble method of machine learning," in *Proc. 2015 Int. Conf. Comput. Commun. Control Automat.*, 2015, pp. 291–295.
- [16] A. Verma and V. Ranga, "ELNIDS: Ensemble learning based network intrusion detection system for RPL based Internet of Things," in *Proc. 2019 4th Int. Conf. Internet Things: Smart Innov. Usages*, 2019, pp. 1–6.
- [17] I. Al-Turaiki and N. Altwaijry, "A convolutional neural network for improved anomaly-based network intrusion detection," *Big Data*, vol. 9, no. 3, pp. 233–252, 2021.
- [18] A. Abbas, M. A. Khan, S. Latif, M. Ajaz, A. A. Shah, and J. Ahmad, "A new ensemble-based intrusion detection system for Internet of Things," *Arabian J. Sci. Eng.*, vol. 47, pp. 1805–1819, 2022.
- [19] H. Rajadurai and U. D. Gandhi, "A stacked ensemble learning model for intrusion detection in wireless network," *Neural Comput. Appl.*, vol. 34, no. 18, pp. 15387–15395, 2022.
- [20] Q. A. Al-Hajja and M. Al-Dala'ien, "ELBA-IoT: An ensemble learning model for botnet attack detection in IoT networks," *J. Sensor Actuator Netw.*, vol. 11, no. 1, 2022, Art. no. 18.
- [21] Y. Cao, Z. Wang, H. Ding, J. Zhang, and B. Li, "An intrusion detection system based on stacked ensemble learning for IoT network," *Comput. Elect. Eng.*, vol. 110, 2023, Art. no. 108836.
- [22] R. Lazzarini, H. Tianfield, and V. Charissis, "A stacking ensemble of deep learning models for IoT intrusion detection," *Knowl.-Based Syst.*, vol. 279, 2023, Art. no. 110941.
- [23] A. Yulianto, P. Sukarno, and N. A. Suwastika, "Improving adaboost-based intrusion detection system (IDS) performance on CIC IDS 2017 dataset," in *Proc. J. Phys.: Conf. Ser.*, 2019, vol. 1192, pp. 1–10.
- [24] S. Phetlasy, S. Ohzahata, C. Wu, and T. Kato, "Applying SMOTE for a sequential classifiers combination method to improve the performance of intrusion detection system," in *Proc. 2019 IEEE Int. Conf. Dependable, Autonomic Secure Comput., Int. Conf. Pervasive Intell. Comput., Int. Conf. Cloud Big Data Comput., Int. Conf. Cyber Sci. Technol. Congr.*, 2019, pp. 255–258.
- [25] A. O. Widodo, B. Setiawan, and R. Indraswari, "Machine learning-based intrusion detection on multi-class imbalanced dataset using smote," *Procedia Comput. Sci.*, vol. 234, pp. 578–583, 2024.
- [26] S. Rajabi, S. Asgari, S. Jamali, and R. Fotuhi, "An intrusion detection system using the artificial neural network-based approach and firefly algorithm," *Wirel. Pers. Commun.*, vol. 137, no. 4, pp. 2409–2440, 2024.
- [27] R. Fotuhi, M. Abdan, and S. Ghasemi, "A self-adaptive intrusion detection system for securing UAV-to-UAV communications based on the human immune system in UAV networks," *J. Grid Comput.*, vol. 20, no. 3, 2022, Art. no. 22.
- [28] M. Mahdavisarif, S. Jamali, and R. Fotuhi, "Big data-aware intrusion detection system in communication networks: A deep learning approach," *J. Grid Comput.*, vol. 19, no. 4, 2021, Art. no. 46.
- [29] P. Pant, A. Kumar, L. K. Vashishtha, S. Dash, N. K. Ray, and S. K. Sahu, "A comparative study of deep learning techniques for network intrusion detection," in *Proc. 2024 Int. Conf. Emerg. Syst. Intell. Comput.*, 2024, pp. 722–727.
- [30] B. Farzaneh, N. Shahriar, A. H. Al Muktafir, M. S. Towhid, and M. S. Khosravi, "DTL-5G: Deep transfer learning-based DDoS attack detection in 5G and beyond networks," *Comput. Commun.*, vol. 228, 2024, Art. no. 107927.
- [31] P. Tiwary, A. Madhubalan, and A. Gautam, "Partitioned problem space (PaPS) ensemble for zero-day intrusion detection," Dec. 2023, doi: [10.21203/rs.3.rs-3781341/v1](https://doi.org/10.21203/rs.3.rs-3781341/v1).
- [32] D. Kumari, A. Sinha, S. Dutta, and P. Pranav, "Optimizing neural networks using spider monkey optimization algorithm for intrusion detection system," *Sci. Rep.*, vol. 14, no. 1, 2024, Art. no. 17196.
- [33] Y. Meidan et al., "N-BaIoT—Network-based detection of IoT botnet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Jul./Sep. 2018.
- [34] P. H. Do, T. D. Le, V. Vishnevsky, A. Berezkin, and R. Kirichek, "A horizontal federated learning approach to IoT malware traffic detection: An empirical evaluation with n-baiot dataset," in *Proc. 2024 26th Int. Conf. Adv. Commun. Technol.*, 2024, pp. 1494–1506.
- [35] R. Kalakoti, S. Nömm, and H. Bahsi, "In-depth feature selection for the statistical machine learning-based botnet detection in IoT networks," *IEEE Access*, vol. 10, pp. 94518–94535, 2022.
- [36] B. Saeed, S. Arshad, S. M. U. Saeed, M. A. Azam, A. Akram, and S. Gul, "Intelligent Internet of Things attack detection: Novel approaches and technologies," in *Proc. 2023 Int. Conf. IT Ind. Technol.*, 2023, pp. 1–6.

- [37] V. Engen, J. Vincent, and K. Phalp, "Exploring discrepancies in findings obtained with the KDD Cup'99 data set," *Intell. Data Anal.*, vol. 15, no. 2, pp. 251–276, 2011.
- [38] T. Su, H. Sun, J. Zhu, S. Wang, and Y. Li, "BAT: Deep learning methods on network intrusion detection using NSL-KDD dataset," *IEEE Access*, vol. 8, pp. 29575–29585, 2020.
- [39] W. Xu, J. Jang-Jaccard, A. Singh, Y. Wei, and F. Sabrina, "Improving performance of autoencoder-based network anomaly detection on NSL-KDD dataset," *IEEE Access*, vol. 9, pp. 140136–140146, 2021.
- [40] S. Chauhan, L. Mahmoud, T. Sheth, S. Gangopadhyay, and A. K. Gangopadhyay, "Generating adversarial examples using LAD," in *Proc. Int. Conf. Intell. Data Eng. Automated Learn.*, 2023, pp. 154–165.
- [41] J. Gu, "An effective intrusion detection model based on pls-logistic regression with feature augmentation," in *Proc. Cyber Secur.: 17th China Annu. Conf.*, 2020, pp. 133–140.
- [42] R. Wazirali, "An imoencoder intrusion detection system based on knn hyperparameter tuning and cross-validation," *Arabian J. Sci. Eng.*, vol. 45, no. 12, pp. 10859–10873, 2020.
- [43] K. Rai, M. S. Devi, and A. Guleria, "Decision tree based algorithm for intrusion detection," *Int. J. Adv. Netw. Appl.*, vol. 7, no. 4, 2016, Art. no. 2828.
- [44] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve bayes feature embedding," *Comput. Secur.*, vol. 103, 2021, Art. no. 102158.
- [45] M. Panda and M. R. Patra, "Network intrusion detection using naive bayes," *Int. J. Comput. Sci. Netw. Secur.*, vol. 7, no. 12, pp. 258–263, 2007.
- [46] S. Mukherjee and N. Sharma, "Intrusion detection using naive bayes classifier with feature reduction," *Procedia Technol.*, vol. 4, pp. 119–128, 2012.
- [47] Y. Chang, W. Li, and Z. Yang, "Network intrusion detection based on random forest and support vector machine," in *Proc. 2017 IEEE Int. Conf. Comput. Sci. Eng. IEEE Int. Conf. Embedded Ubiquitous Comput.*, 2017, vol. 1, pp. 635–638.
- [48] I. Ahmad, M. Basher, M. J. Iqbal, and A. Rahim, "Performance comparison of support vector machine, random forest, and extreme learning machine for intrusion detection," *IEEE Access*, vol. 6, pp. 33789–33795, 2018.
- [49] P. Su, Y. Liu, and X. Song, "Research on intrusion detection method based on improved smote and SGBost," in *Proc. 8th Int. Conf. Commun. Netw. Secur.*, 2018, pp. 37–41.
- [50] I. Syarif, E. Zaluska, A. Prugel-Bennett, and G. Wills, "Application of bagging, boosting and stacking to intrusion detection," in *Proc. 8th Int. Conf. Mach. Learn. Data Mining Pattern Recognit.*, 2012, pp. 593–602.
- [51] J. Brownlee, "How to choose a feature selection method for machine learning," *Mach. Learn. Mastery*, vol. 10, pp. 1–7, 2019.
- [52] K. Dissanayake and M. G. Md Johar, "Comparative study on heart disease prediction using feature selection techniques on classification algorithms," *Appl. Comput. Intell. Soft Comput.*, vol. 2021, no. 1, 2021, Art. no. 5581806.
- [53] N. Bulso, M. Marsili, and Y. Roudi, "On the complexity of logistic regression models," *Neural Comput.*, vol. 31, no. 8, pp. 1592–1623, 2019.
- [54] H. M. Sani, C. Lei, and D. Neagu, "Computational complexity analysis of decision tree algorithms," in *Proc. Artif. Intell. XXXV: 38th SGAI Int. Conf. Artif. Intell.*, 2018, pp. 191–197.
- [55] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Trans. Inf. Theory*, vol. IT-13, no. 1, pp. 21–27, Jan. 1967.
- [56] M. Boullé, "Compression-based averaging of selective naive Bayes classifiers," *J. Mach. Learn. Res.*, vol. 8, pp. 1659–1685, 2007.
- [57] I. W. Tsang, J. T. Kwok, and N. Cristianini, "Core vector machines: Fast SVM training on very large data sets," *J. Mach. Learn. Res.*, vol. 6, no. 4, 2005.
- [58] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, pp. 5–32, 2001.
- [59] E. Scornet, G. Biau, and J.-P. Vert, "Consistency of random forests," *Ann. Statist.*, vol. 43, no. 4, pp. 1716–1741, 2015.
- [60] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 785–794.
- [61] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992.
- [62] S. A. Elsaid, E. Shehab, A. M. Mattar, A. T. Azar, and I. A. Hameed, "Hybrid intrusion detection models based on gwo optimized deep learning," *Discover Appl. Sci.*, vol. 6, no. 10, 2024, Art. no. 531.
- [63] A. T. Azar, E. Shehab, A. M. Mattar, I. A. Hameed, and S. A. Elsaid, "Deep learning based hybrid intrusion detection systems to protect satellite networks," *J. Netw. Syst. Manage.*, vol. 31, no. 4, 2023, Art. no. 82.
- [64] K. Jiang, W. Wang, A. Wang, and H. Wu, "Network intrusion detection combined hybrid sampling with deep hierarchical network," *IEEE Access*, vol. 8, pp. 32464–32476, 2020.
- [65] Y. Yin et al., "IGRF-RFE: A hybrid feature selection method for MLP-based network intrusion detection on UNSW-NB15 dataset," *J. Big data*, vol. 10, no. 1, 2023, Art. no. 15.
- [66] M. Umair, W.-H. Tan, and Y.-L. Foo, "Optimized 1D convolutional neural network for efficient intrusion detection in IoT networks," in *Proc. 2024 IEEE 8th Int. Conf. Signal Image Process. Appl.*, 2024, pp. 1–6.
- [67] L. C. Guimarães and R. S. Couto, "A performance evaluation of neural networks for BotNet detection in the Internet of Things," *J. Netw. Syst. Manage.*, vol. 32, no. 4, 2024, Art. no. 98.
- [68] N. G. Pardeshi and D. V. Patil, "Applying gini importance and RFE methods for feature selection in shallow learning models for implementing effective intrusion detection system," in *Proc. Int. Conf. Appl. Mach. Intell. Data Analytics*, 2023, pp. 214–234.
- [69] N. G. Pardeshi and D. V. Patil, "Binary and multiclass classification intrusion detection system using benchmark NSL-KDD and machine learning models," in *Proc. 2024 Int. Conf. Data Sci. Netw. Secur.*, 2024, pp. 1–7.
- [70] T. Wu, H. Fan, H. Zhu, C. You, H. Zhou, and X. Huang, "Intrusion detection system combined enhanced random forest with smote algorithm," *EURASIP J. Adv. Signal Process.*, vol. 2022, no. 1, 2022, Art. no. 39.



LOREEN MAHMOUD received the bachelor's degree in computer engineering and automatic control from Latakia University (Formerly Tishreen University), Latakia, Syria, in 2017, and the master's degree in cyber security from JAIN university, Bangalore, India, in 2021. She is currently working toward the Ph.D. degree with the Department of Computer Science and Engineering, Indian Institute of Technology Roorkee, Roorkee, India.



MADHUSANKA LIYANAGE (Senior Member, IEEE) is currently an Associate Professor/Ad Astra Fellow and the Director of Network Softwarization and Security Labs (NetsLab) with the School of Computer Science, University College Dublin, Dublin, Ireland. He is also acting as a Docent/Adjunct Professor with the University of Oulu, Oulu, Finland, University of Ruhuna, Matara, Sri Lanka and University of Sri Jayawardenepura, Sri Lanka. His research interests include 5G/6G, SDN, IoT, Blockchain, MEC, mobile, and virtual network security. Dr. Liyanage's was the recipient of the prestigious Marie Skłodowska-Curie Actions Individual Fellowship and Government of Ireland Postdoctoral Fellowship during 2018–2020, in 2020, "2020 IEEE ComSoc Outstanding Young Researcher" Award by IEEE ComSoc EMEA, during 2021–2024, he was ranked among the World's Top 2% Scientists (2020,2021,2022,2023) in the List prepared by Elsevier BV, Stanford University, USA, an Irish Research Council (IRC) Research Ally Prize as part of the IRC Researcher of the Year 2021, 2023 and 2024 Awards for the positive impact he has made as a Supervisor.



JITIN SINGLA received the master's and Ph.D. degrees from the University of Southern California, Los Angeles, CA, USA, in 2000. He is currently an Assistant Professor in the Department of Biosciences and Engineering and Computer Science and Engineering of the Indian Institute of Technology Roorkee, Roorkee, India. He is also in Computational Biology and Pattern recognition in Medical data.



SUGATA GANGOPADHYAY received the masters in high performance computing and simulation and Ph.D. in computational biology and bioinformatics from the University of Southern California in 2020. He has taught with IIT Kharagpur, the Indian Statistical Institute, Kolkata, India, and Birla Institute of Technology And Science - Pilani, India. He is currently a Professor with the Department of Computer Science and Engineering of the Indian Institute of Technology Roorkee, Roorkee, India. His research interests include in information and network security and quantum computing.